L14: Finite-Capacity Queues and Blocking

Simulation Without a Black Box

Outline

Learning Objectives

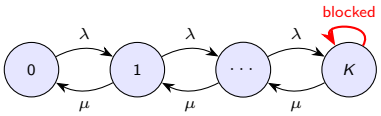
By the end of this lecture you will be able to:

- Describe the M/M/1/K model and explain what blocking means.
- Derive the truncated geometric stationary distribution for M/M/1/K.
- Compute the blocking probability P_K and effective throughput λ_{eff}.
- Implement finite-buffer blocking in SimPy using a queue-length check.
- Analyse the trade-off between buffer size K, blocking rate, and congestion.

The M/M/1/K Model: Finite Buffer

M/M/1/K notation

- Poisson arrivals at rate λ
- Exponential service at rate μ (single server)
- Maximum system capacity K (server + queue)
- Arriving customer finding K in system is **blocked** (lost)
- No stability condition needed: system is always finite



Real examples: parking lots, call centres with hold queues, hospital beds, buffer in manufacturing line.

Balance Equations and Truncated Geometric Distribution

Balance in state n ($0 \leq n \leq K$): same as M/M/1,

$$\pi_n = \rho^n \pi_0, \quad \rho = \lambda/\mu$$

(no stability restriction — ρ may be ≥ 1).
Normalisation over $\{0, \ldots, K\}$:

$$\pi_0 = \frac{1-\rho}{1-\rho^{K+1}} \quad (\rho \neq 1), \qquad \pi_0 = \frac{1}{K+1} \quad (\rho = 1)$$

Blocking probability

$$P_K = \pi_K = \frac{(1-\rho)\rho^K}{1-\rho^{K+1}}$$

$$\lambda_{\text{eff}} = \lambda(1 - P_K) \quad \text{(effective throughput)}$$

As $K \to \infty$: $P_K \to 0$ and we recover M/M/1 (if $\rho < 1$).

SimPy: Checking Queue Length Before Requesting

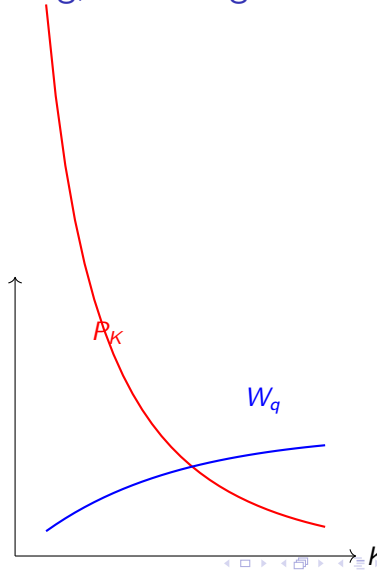
```
import simpy, numpy as np

def customer(env, server, K, rng, stats):
    # Check: server.count + len(server.queue) == current occupancy
    if server.count + len(server.queue) >= K:
        stats["blocked"] += 1
        return                # customer is lost
    stats["arrived"] += 1
    arrive = env.now
    with server.request() as req:
        yield req
        stats["waits"].append(env.now - arrive)
        yield env.timeout(rng.exponential(1.0))  # mu=1

def source(env, server, K, lam, rng, stats):
    while True:
        yield env.timeout(rng.exponential(1/lam))
        env.process(customer(env, server, K, rng, stats))

K, lam = 5, 0.9
stats = {"arrived": 0, "blocked": 0, "waits": []}
```

Trade-off: Buffer Size, Blocking, and Congestion

At $\rho = 0.8$, $\mu = 1$:

K	P_K	λ_{eff}	W_q
1	0.444	0.445	0.000
2	0.262	0.590	0.222
5	0.066	0.747	1.483
10	0.007	0.794	3.291
∞	0.000	0.800	4.000

Small K: low congestion but high loss. Large K: full M/M/1 behaviour.



Effective Throughput and Little's Law in M/M/1/K

Little's Law still applies to the admitted customers

$$L = \lambda_{\text{eff}} \cdot W = \lambda(1 - P_K) \cdot W$$

where W is the mean sojourn time *conditional on admission*.

- If the goal is maximising throughput with $P_K \leq 0.05$, choose the smallest K satisfying the constraint.
- If the goal is minimising congestion, small K helps — but at the cost of lost customers.
- In manufacturing: a blocked customer is a *starved upstream stage* — blocking propagates.

Do not confuse utilisation with throughput

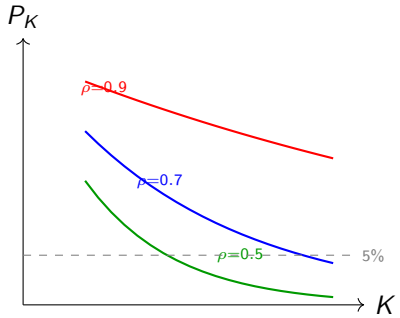
At high ρ and small K, the server is highly utilised — but effective throughput λ_{eff} can be much less than λ.

Plot: Blocking Probability P_K vs. Buffer Size K

- Each curve is a different load level ρ.
- At high ρ, large K is needed to drive blocking below 5%.
- At $\rho = 0.5$, even $K = 4$ achieves $P_K < 1\%$.

Design rule

Choose K such that $P_K \leq$ target blocking rate. Use the analytical formula for M/M/1/K; validate with simulation for general service times.



Key Takeaways

Core ideas from L14

1. M/M/1/K truncates the geometric distribution: $\pi_n = \rho^n \pi_0$ for $n = 0, \ldots, K$.
2. **Blocking probability** P_K decreases exponentially with K at fixed ρ.
3. **Effective throughput** $\lambda_{\text{eff}} = \lambda(1 - P_K)$ — not the offered load λ.
4. SimPy blocking: check `server.count + len(server.queue) >= K` before requesting.
5. Small K reduces wait times at the cost of turning customers away — choose K from a service-level constraint.

Next: L15 — Reneging and customer impatience: timeout-race pattern in SimPy.